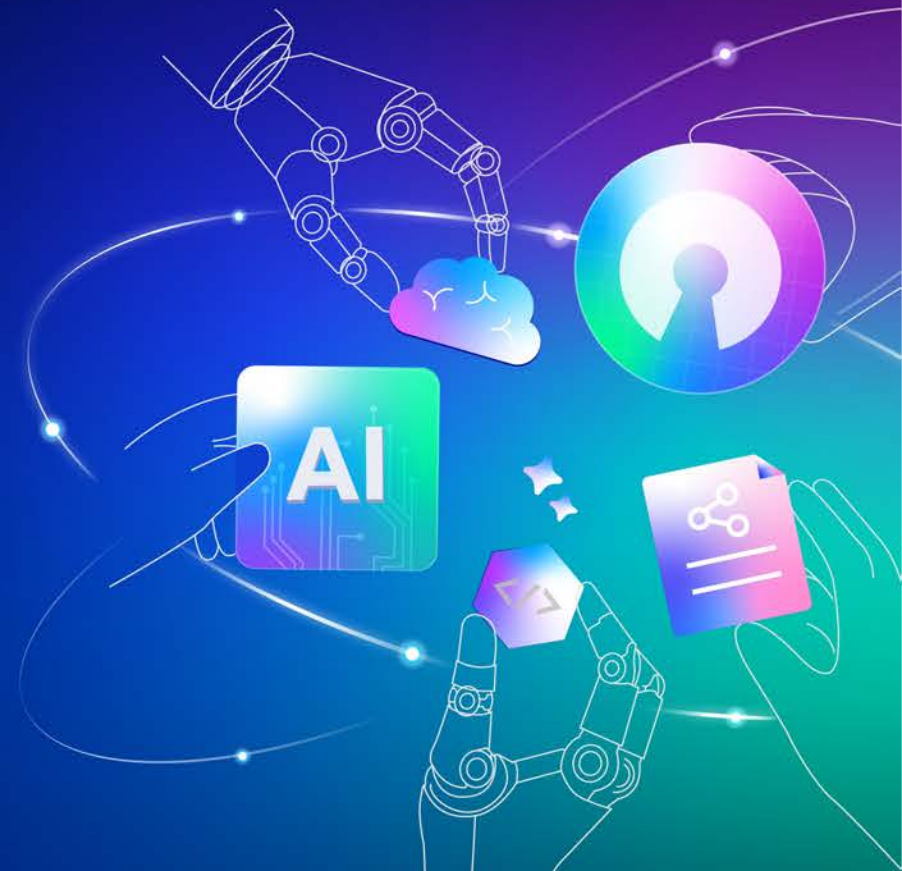


# AI on Rails

좋은 제약이 더 큰 자유를 만든다

강영묵 | NST 국가과학AI연구센터

OST2026



# 서울에서 부산, 보통 무엇을 타고 가시나요?

주로 이용하는 교통수단은 무엇일까요?



자동차



버스



기차



# 같은 부산행, 맡기는 일이 다릅니다

직접 고르는 일과 다른 사람에게 맡기는 일

---

## 자동차

직접 운전하며  
경로를 선택

---

## 버스

정해진 노선의  
운행을 맡김

---

## 기차

정해진 선로의  
운행을 맡김



# 운전면허 없이도, 빠른 이동을 이용할 수 있습니다

---

행선지와 시간 선택



승차권을 구매 탑승

승객이 고속 운전 기술까지 익힐 필요는 없습니다



# 기차의 안전을 운영 시스템이 함께 만듭니다

## 신호와 제동

위험한 운행을 제한

## 차량과 시설 점검

고장을 미리 발견

## 전문 운영

운영과 이상 상황에 대응

승객 개인의 운전 실력에 기대는 부담을 줄입니다



# 시속 300km 자동차를 상상해 보면

가상의 최고 속도 사례

# 300

km/h

---

최고 속도만으로는  
안전하게 반복할 수 있는 방법이 되지 않습니다



# 자유롭게 생성한 결과가 원하는 것과 다르다면?

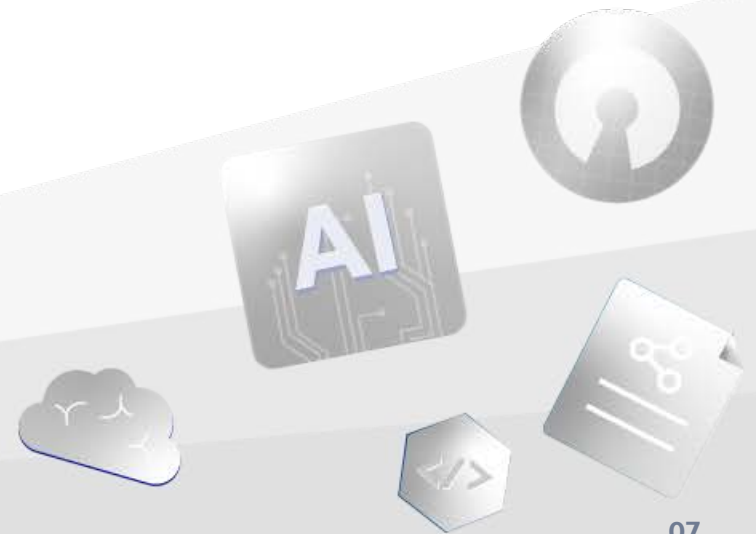
**처음 요청**

**“이 요구사항대로 앱을 만들어 줘”**

**수정 요청**

**원인을 찾고, 다른 부분을 유지하며 고치기**

**수정에는 구조와 영향 범위를 이해하는 역량이 필요할 수 있습니다**



# 좋은 제약은 고칠 범위를 좁혀 줍니다

색상을 바꾸는 수정이라면

## 자유 형식의 전체 코드

---

어디에 색을 지정했는지  
구조부터 파악

## 이름이 정해진 색상 토큰

---

정해 둔 항목을 바꾸고  
영향받는 화면을 확인

변경 지점이 명확하면, 수정과 검증의 출발점도 명확해집니다



**전문성을 미리 담아 두면,  
이용자가 익혀야 할 것이 줄어듭니다**

---

**철도에는 운행 체계를,  
AI 작업 환경에는 규약과 검증 도구를**



# Rails의 CoC도 반복할 결정을 미리 정합니다

Convention over Configuration · 설정보다 규약

## 부산행 기차

---

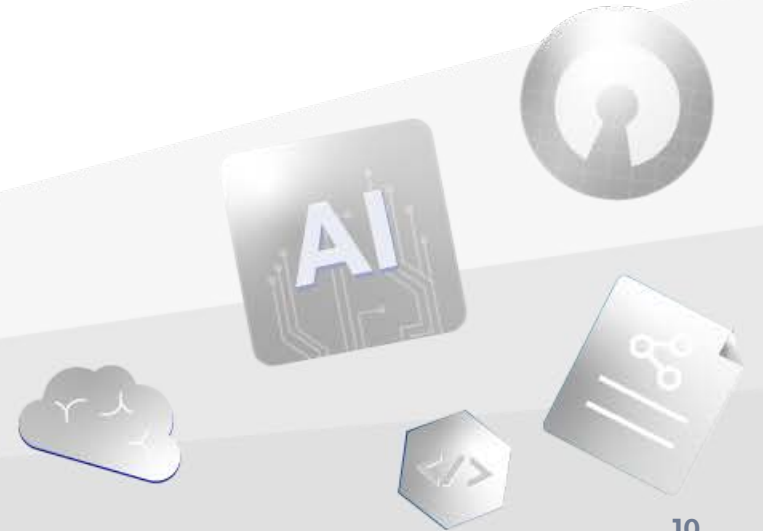
어느 열차를 탈지는 선택  
매 구간의 운행은 맡김

## Rails

---

만들 서비스는 직접 설계  
반복 설정은 규약에 맡김

CoC는 반복 설정을 줄이고, 검사와 권한은 별도로 설계합니다



# 같은 AI인데 결과는 왜 달라질까?

---

모델이 같아도 도구, 표현, 규칙, 권한이 달라지면 결과가 달라집니다.

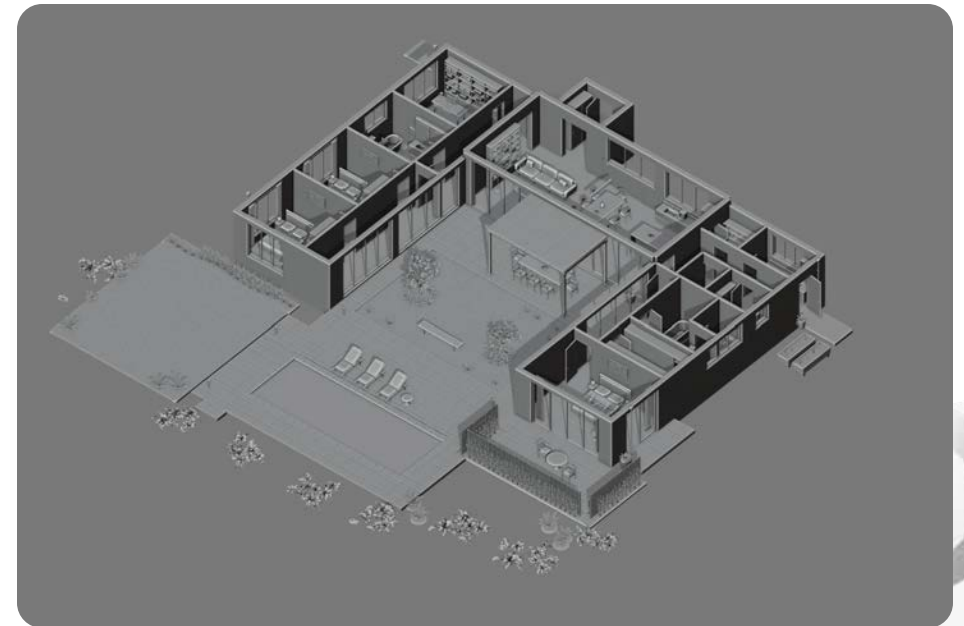


# Astra와 Blender로 만든 주택

OpenAI 공개 사례 · Thomas Ricouard · 2026.09.04



**외관 렌더**

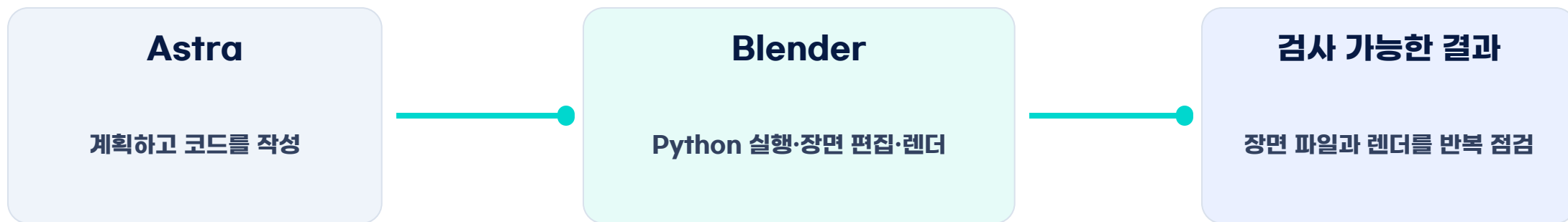


**구조를 확인하는 모델 화면**

출처: OpenAI Developers · Architectural visualization with Astra

# Astra가 Blender를 만났을 때

언어모델의 계획이 전문 도구를 통해 편집 가능한 3D 작업으로 이어졌습니다



모델과 전문 도구가 함께 결과를 만듭니다



# 화합물 구조 검색을 요청했습니다

“화합물 구조를 검색하는 파이썬 코드를 만들어줘”

## 생성된 코드

```
python Copy code

import pubchempy as pcp

# 검색어 입력
compound_name = input("검색할 화합물 이름을 입력하세요: ")

# PubChem 데이터베이스에서 검색
results = pcp.get_compounds(compound_name, 'name')

# 결과 출력
for result in results:
    print(result.iupac_name)
```

## 이름 검색 결과

```
▶ import pubchempy as pcp

# 검색어 입력
compound_name = "aspirin"

# PubChem 데이터베이스에서 검색
results = pcp.get_compounds(compound_name, 'name')

# 결과 출력
for result in results:
    print(result.iupac_name)
```

2-acetoxybenzoic acid

# RDKit을 지정해 다시 요청했습니다

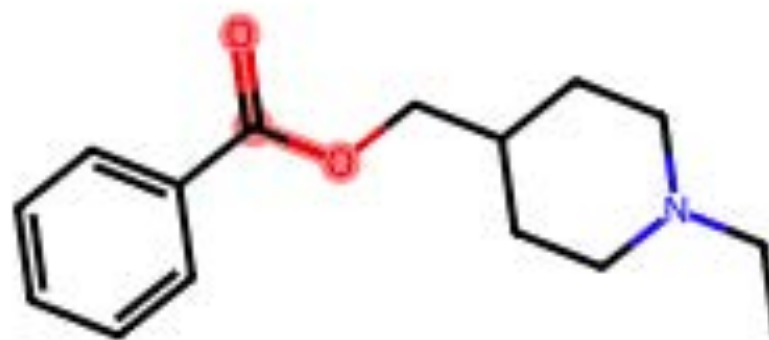
“RDKit으로 화합물 구조 검색하는 파이썬 코드 만들어줘”

## 생성된 코드

이제 아래 코드를 사용하여 화합물 구조 검색을 수행할 수 있습니다.

```
python Copy code  
  
from rdkit import Chem  
from rdkit.Chem import Draw  
  
# 검색할 화합물 구조의 SMILES 문자열  
smiles = "C1=CC=C(C(=C1)C(=O)OCC2CCN(CC2)CC)"  
  
# SMILES 문자열을 RDKit 문자 객체로 변환  
mol = Chem.MolFromSmiles(smiles)  
  
# 검색할 서브구조의 SMILES 문자열  
substructure_smiles = "C(=O)O"  
  
# SMILES 문자열을 RDKit 문자 객체로 변환  
substructure = Chem.MolFromSmiles(substructure_smiles)  
  
# 서브구조 검색  
matches = mol.GetSubstructMatches(substructure)  
  
# 검색된 서브구조의 인덱스를 출력  
print(matches)
```

## 부분 구조 검색 결과



# 한 단어가 선택 공간을 바꿨습니다

화합물 구조 검색을 요청한 실제 발표자 경험

**화합물 구조 검색  
코드를 만들어줘**

원하는 방향과 차이

**RDKit으로 구조 검색  
코드를 만들어줘**

부분 구조 검색과 이미지 표현

**도구 이름이 문제의 언어를 정했습니다**



# AI가 만든 웹 앱의 실행 기반

앱을 실행하는 코드 아래에는 브라우저가 있습니다

## AI가 작성한 앱 코드

HTML · CSS · JavaScript

새로 만드는 부분

## 브라우저 / 웹 플랫폼

화면 표시 · 입력 처리 · 코드 실행

재사용하는 기반



# AI도 웹의 규칙을 사용합니다

저장 버튼 하나를 만들 때도 같은 규칙이 적용됩니다

```
<button type="button">저장</button>
```

---

## HTML

버튼의 의미

---

## CSS

모양과 배치

---

## JavaScript

클릭 후 동작

브라우저가 이해하는 언어와 기능으로 조합합니다



# 기반이 맡는 일, 앱이 책임질 일

공통 기능을 재사용할수록 앱의 요구사항에 집중할 수 있습니다

## 웹 플랫폼

화면을 그리고  
입력을 처리합니다

## 앱 개발

저장할 데이터와  
권한을 정합니다

기반의 안정성과 앱의 정확성은 별개입니다

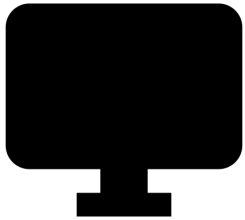


# 기기마다 운영체제가 있습니다

PC, 태블릿, 스마트폰에서 출발합니다

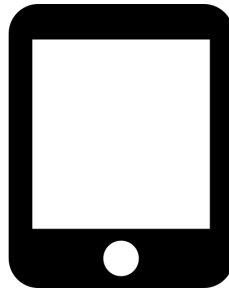
## 운영체제

Windows · macOS · Linu  
x



PC

iPadOS · Android



태블릿

iOS · Android



스마트폰

대표적인 기기와 운영체제 조합의 예



# 운영체제 위에서 브라우저가 실행됩니다

Chrome 같은 브라우저가 웹 프로그램의 실행 환경을 제공합니다



**웹 브라우저 · Chrome 등**

**HTML · CSS · JavaScript와 웹 API**

**Windows · macOS · Linux**

**PC**

**iPadOS · Android**

**태블릿**

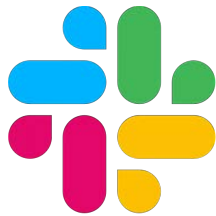
**iOS · Android**

**스마트폰**

Chrome은 대표 예시입니다. Safari·Firefox 등 다른 브라우저도 있습니다

# 익숙한 앱도 이 기반 위에서 작동합니다

브라우저에서 사용하는 웹 버전의 예



**Slack**

업무 소통



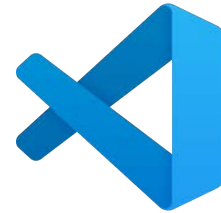
**Discord**

커뮤니티와 대화



**Notion**

문서와 협업



**Visual Studio Code**

VS Code for the Web

**웹 브라우저 · Chrome 등**

**운영체제**

**PC 등 각 서비스가 지원하는 기기**

기기·브라우저별 지원 기능은 다를 수 있습니다

# AI와 만드는 앱은 웹에만 머물지 않습니다

JavaScript 개발 생태계를 여러 형태의 앱에 활용합니다

## Node.js와 npm을 활용한 개발 환경

라이브러리 재사용과 앱 빌드

### 웹 서비스

브라우저에서 이용

### 데스크톱 앱

PC에 설치

### 스마트폰 앱

휴대폰에 설치

앱의 목적과 업무 규칙은  
이 기반 위에 구현합니다



# 웹 기술로 만드는 데스크톱 앱

Electron은 실행 기반을 앱과 함께 제공합니다



Slack



Discord



Notion



VS Code Desktop

## Electron

### Chromium

HTML · CSS · JavaScript로 UI 표시

### Node.js

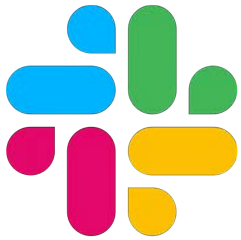
파일과 시스템 기능 활용

## 웹 기술을 활용하는 설치형 앱

지원 운영체제는 제품마다 다릅니다

# 같은 서비스가 스마트폰 앱으로도 이어집니다

웹과 데스크톱에서 본 서비스의 모바일 버전



**Slack**



**Discord**



**Notion**

**Android · iOS에 설치해 이용**

**같은 서비스라도 구현 방식은 달라질 수 있습니다**

앱 제공 범위의 예시이며, 동일한 프레임워크 사용을 뜻하지 않습니다

# 웹 개발 도구를 모바일 앱에도 활용합니다

개발 도구는 공유하고, 실행 방식은 목적에 맞게 고릅니다

Node.js와 npm · 프로젝트 생성과 빌드

웹 UI 재사용

Capacitor

WebView + 기기 기능 연결

네이티브 UI 활용

React Native

Android · iOS의 UI 구성요소

웹 UI를 앱 안에서 재사용

개발 도구와 스마트폰 안의 실행 환경은 구분합니다



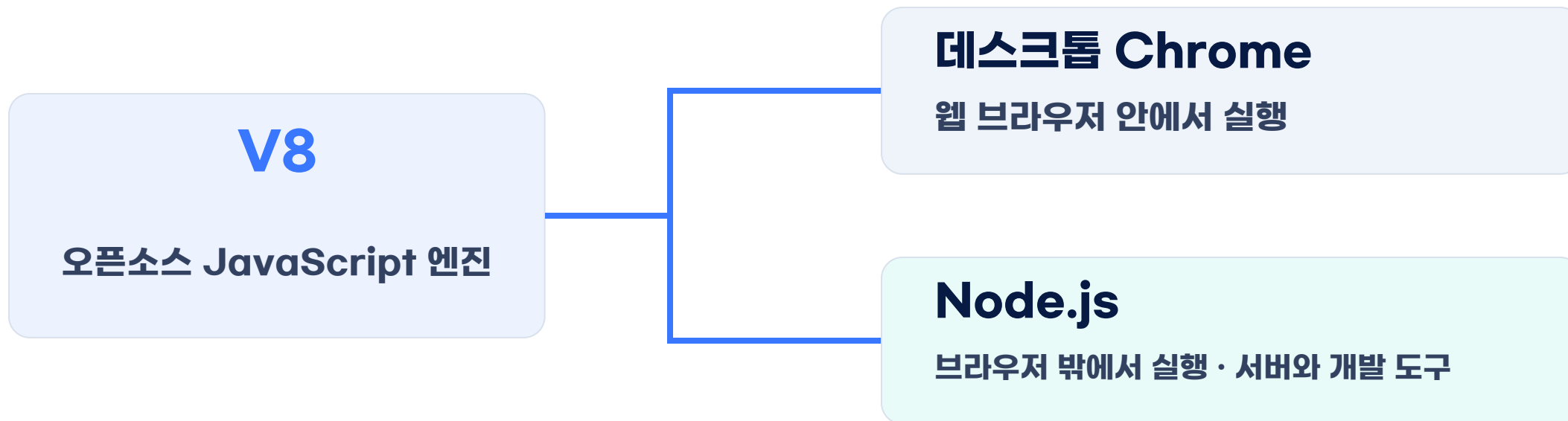
Discord

React Native 적용



# 오픈소스가 실행 기반을 제공합니다

그 기반의 한 사례: V8을 활용하는 실행 환경



**AI는 이미 구현된 실행 기반을 활용합니다**



# AI도 축적된 기술 위에서 앱을 만듭니다

실행 기반은 재사용하고, 앱의 구조와 수정 규칙은 설계합니다.



# 생성은 쉬워졌습니다

---

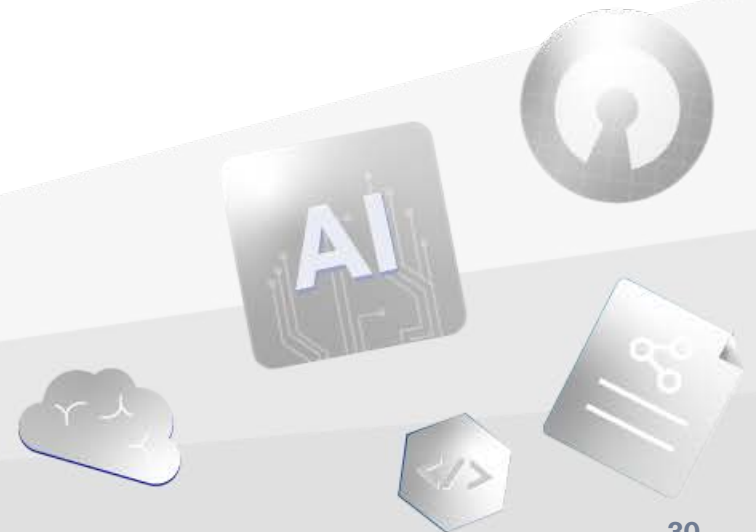
자연어만으로 첫 화면과 다양한 시안을 빠르게 만들 수 있었습니다.



# 수정과 일관성은 다른 문제였습니다

작은 변경이 전체 재생성으로 번지는 순간 비용이 커집니다

|       |      |                   |
|-------|------|-------------------|
| 작은 수정 | ———— | 프롬프트를 다시 입력       |
| 부분 편집 | ———— | 주변 레이아웃까지 변화      |
| 반복 생성 | ———— | 디자인 문맥을 다시 제공     |
| 검증    | ———— | 접근성과 규칙 준수를 별도 확인 |



# 질문을 바꾸었습니다

더 좋은 프롬프트보다, AI가 매번 결정하지 않아도 되는 구조

AI가 HTML을 더 잘 쓰게 하려면? —

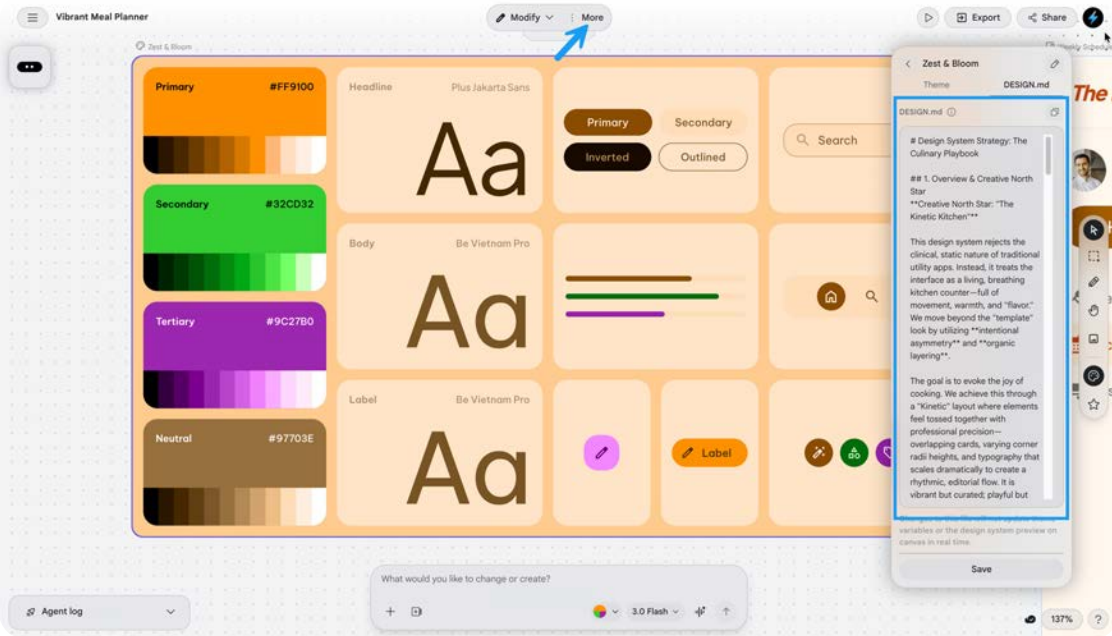
AI가 HTML을 직접 쓸  
필요가 없게 하려면?

생성 문제를 구조 문제로 바꾸었습니다

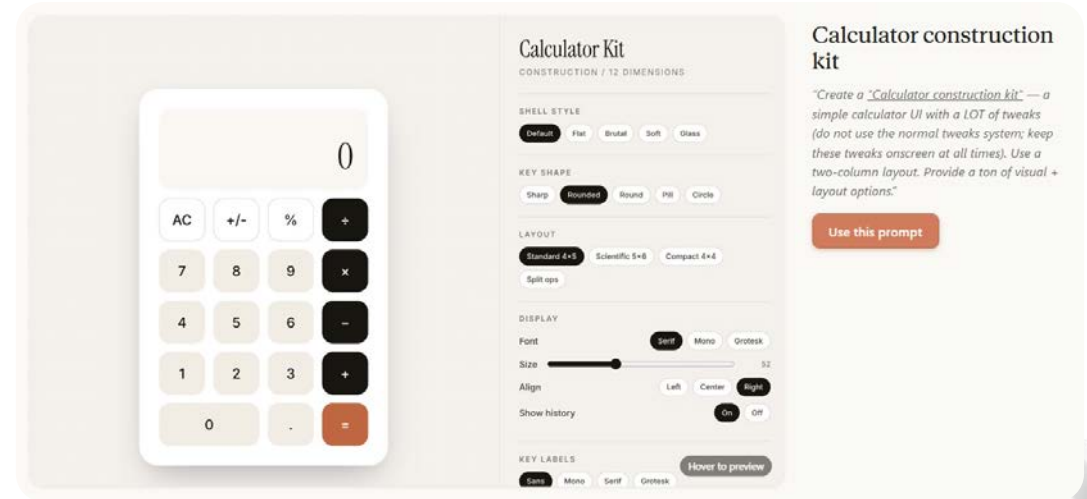


# 먼저 규칙을 만들고, 그 위에서 생성합니다

## Stitch와 Claude Design의 디자인 시스템 접근



Stitch · DESIGN.md



Claude Design · Team Design System



# Conjure-UI

이 고민을 UI 라이브러리로  
구현하고 있습니다

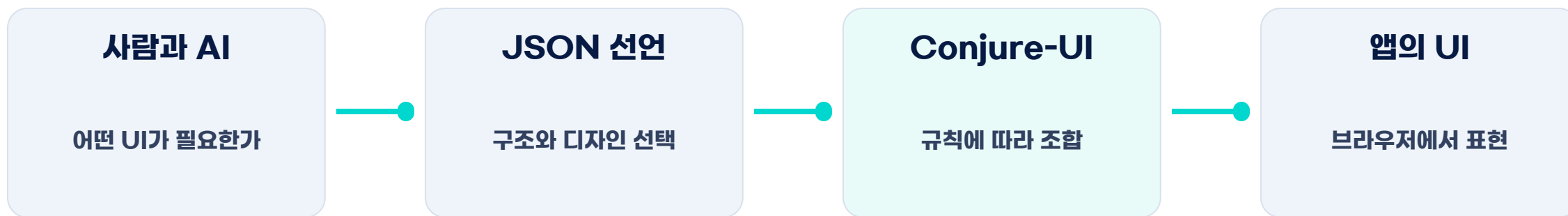
---

제가 직접 설계하고 개발 중인 프로젝트

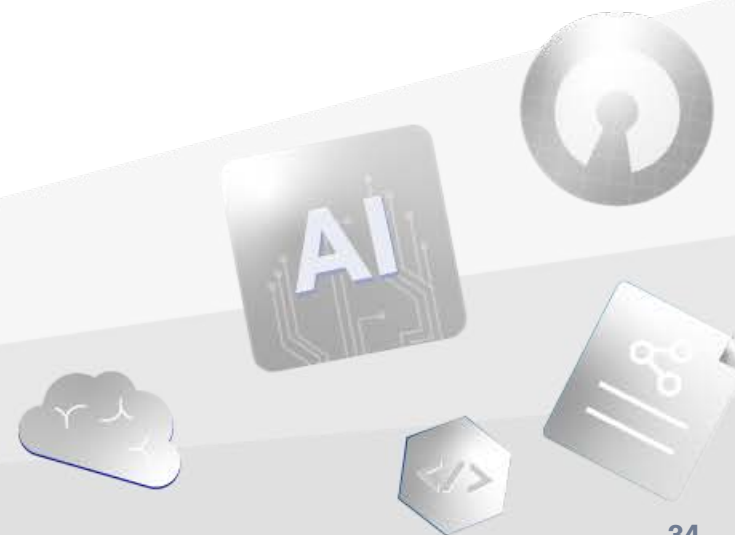


# Conjure-UI는 UI 설계 데이터를 조합합니다

앱의 배치와 외형을 데이터로 정하고, 라이브러리가 UI로 바꿉니다



한 번 정한 구조를 불러오고, 필요한 부분을 바꿉니다



# 출발점은 UI를 데이터로 관리하려던 구상

사람의 편집 도구에서, AI도 다룰 수 있는 표현으로 확장합니다

이전 구상

**Web Builder**

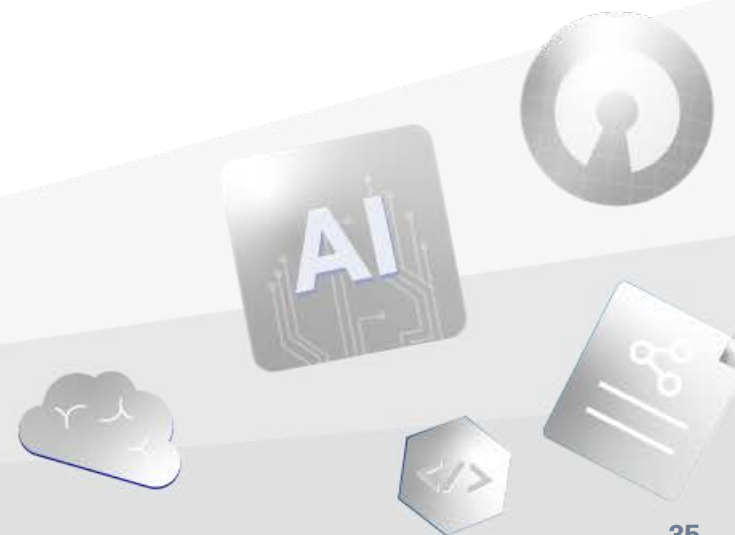
사람이 GUI로 편집

**중간표현**

JSON / XML / YAML

**소스 코드**

**Conjure-UI는 이 중간표현을  
AI와 함께 다루는 UI 선언으로 발전시킵니다**



# Conjure-UI의 구성

Realm에 구조와 디자인을 담고, Materialization으로 UI를 만듭니다

## Realm

UI 하나를 설명하는 모델

### Region

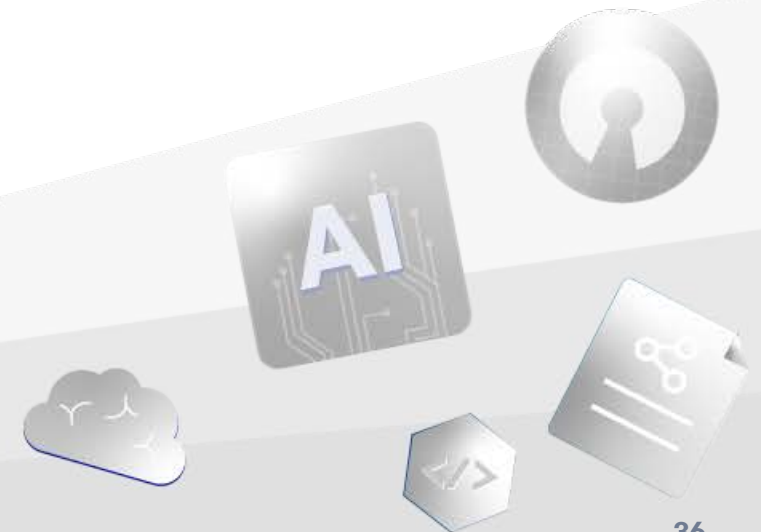
어떤 영역을 어디에 둘까

### Rune

어떤 디자인 값을 쓸까

## Materialization

설계 데이터를  
실제 웹 UI로 변환



# Realm은 UI를 네 부분으로 나눕니다

바꾸려는 부분을 따로 다루기 위한 분리입니다

**구조**

structure

영역과 배치

**외형**

glamour

색상과 글꼴

**동작**

behavior

고정과 이동

**접근성**

accessibility

고대비와 모션 감소



# “색상만 바꾸고 싶다면?”

외형 데이터를 바꾸고, 나머지 설정은 유지합니다

구조

설정 유지

외형

색상 교체

동작

설정 유지

접근성

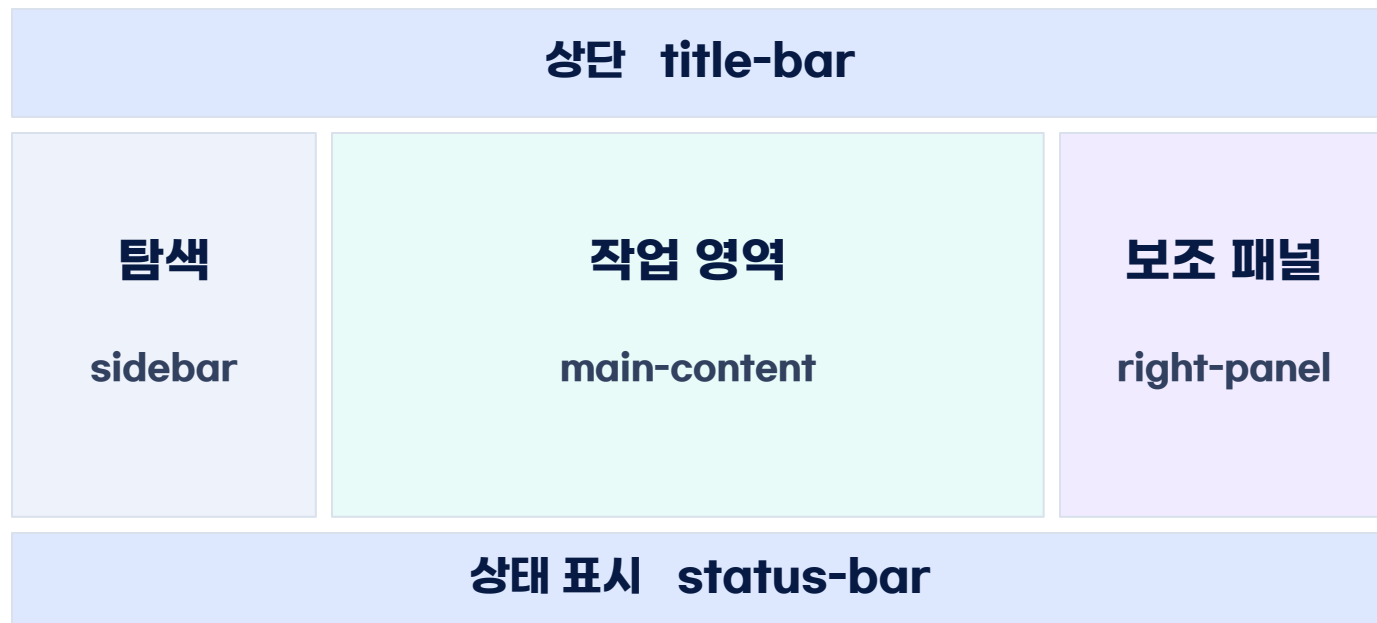
설정 유지

무엇을 바꿀지와 무엇을 유지할지를 구분합니다



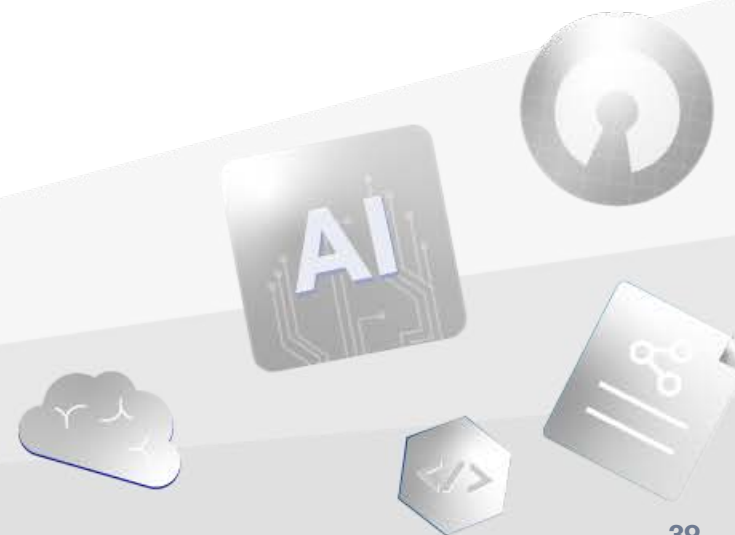
# Region은 앱의 UI를 영역으로 나눕니다

이름과 크기, 보이는 영역의 조합으로 앱의 골격을 정합니다



**같은 영역  
다른 조합**

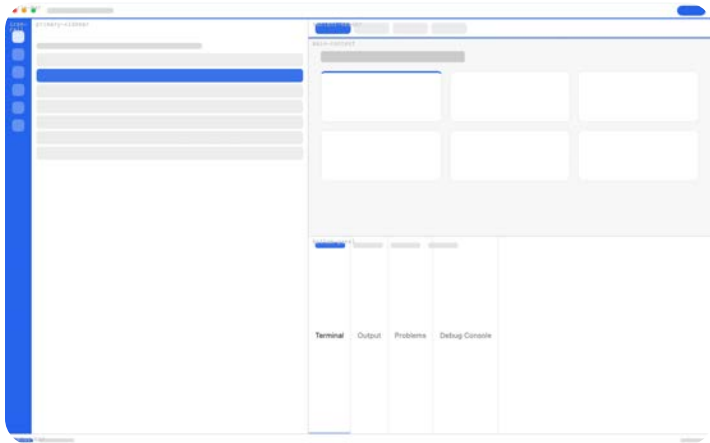
영역 구성 개념도



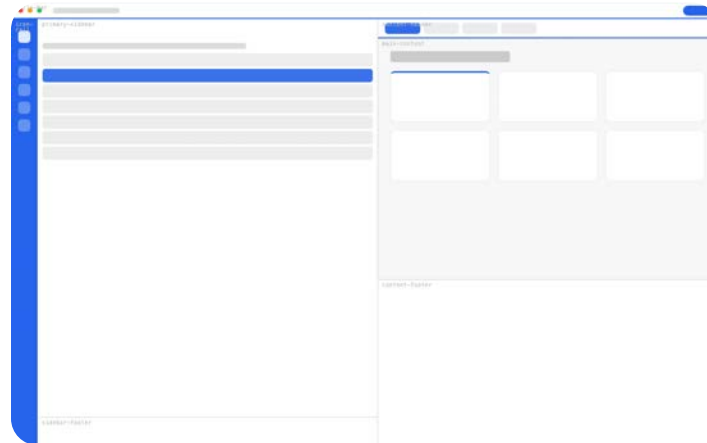
# 프리셋은 앱 골격의 출발점입니다

원자료의 8개 레이아웃 중 IDE, Chat, Document

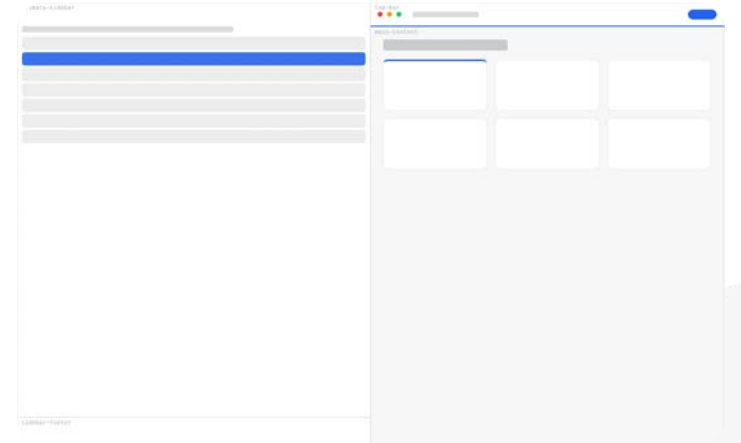
## IDE



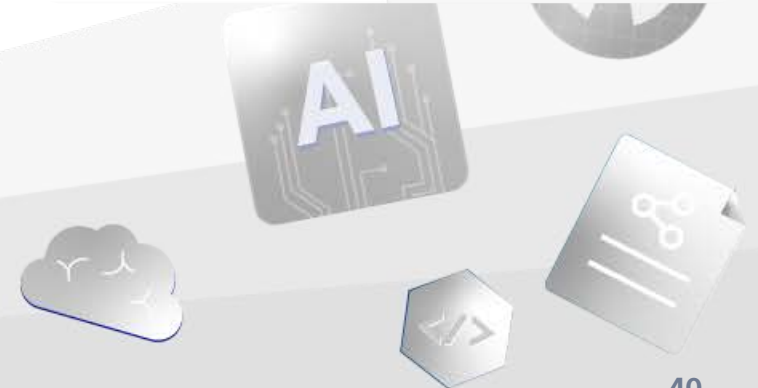
## Chat



## Document



빈 UI부터 매번 새로 정할 필요가 줄어듭니다



# Rune은 재사용할 디자인 값입니다

색상이나 간격을 이름 있는 데이터로 관리합니다

---

## 색상

ColorRune

---

## 글꼴

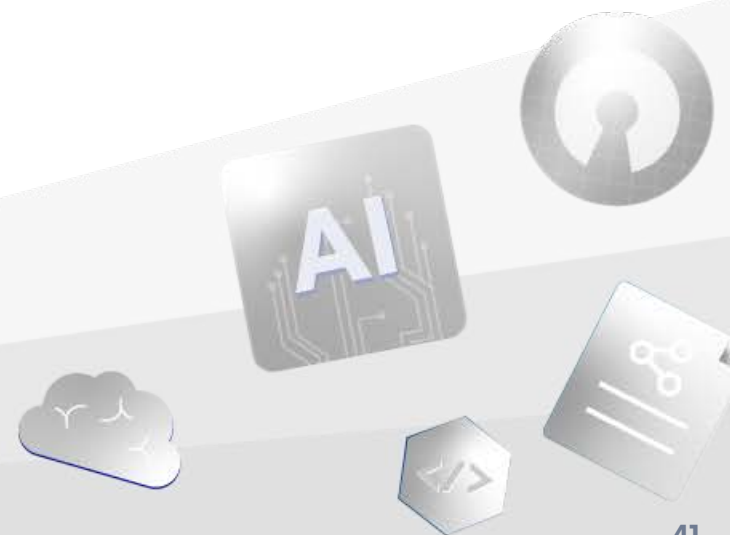
TypographyRune

---

## 간격

SpacingRune

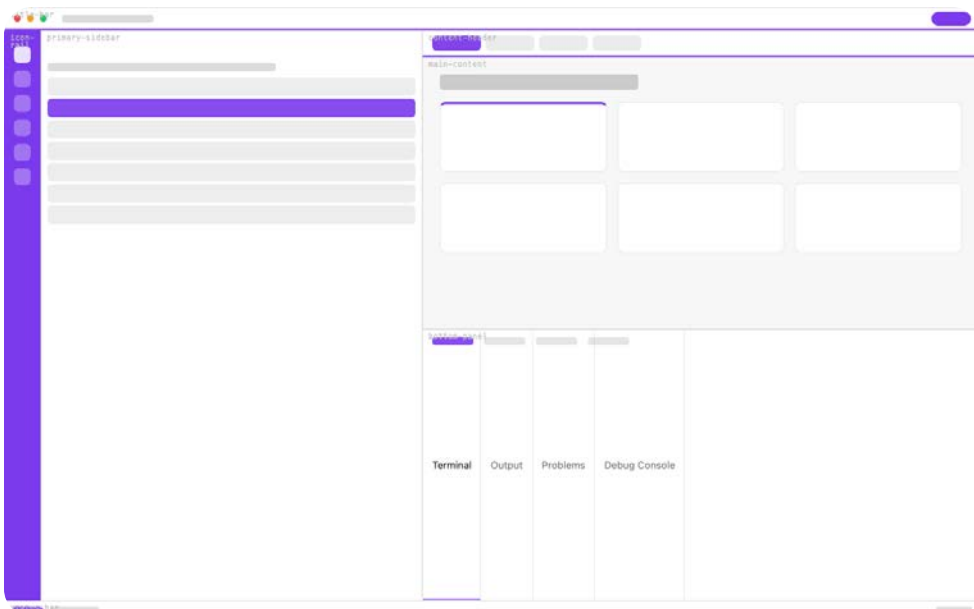
필요한 값을 교환하고, 묶어서 테마에 재사용합니다



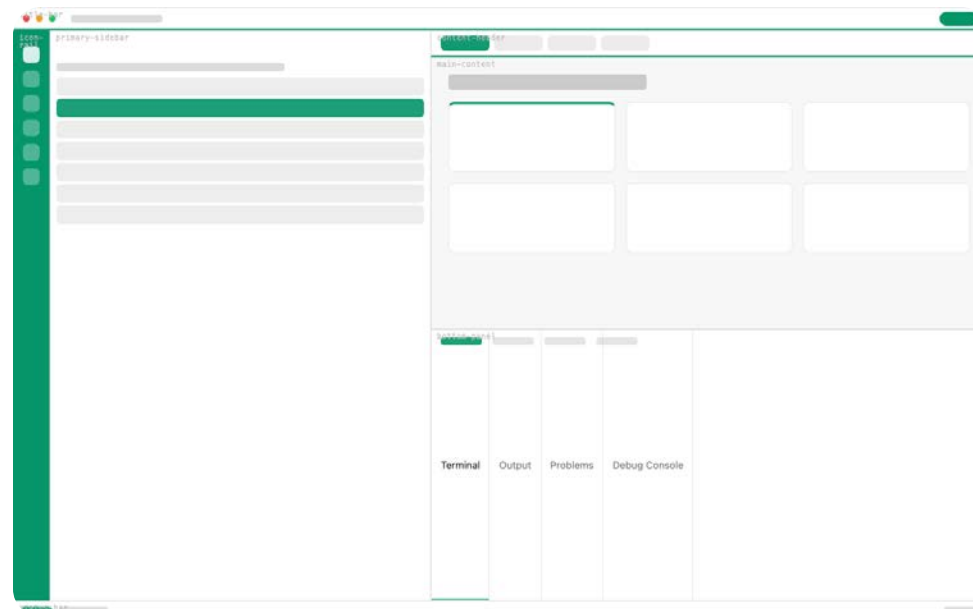
# 구조를 유지한 채 색상을 교환합니다

원자료의 동일 IDE 레이아웃, 서로 다른 색상 Rune

## Violet



## Emerald



# JSON에는 UI의 선택을 적습니다

어떤 골격, 어떤 외형, 어떤 옵션을 사용할지 선언합니다

```
{  
  "preset": "ide",  
  "theme": { "mode": "dark", "accent": "violet" },  
  "structure": { "rightPanel": false },  
  "accessibility": { "reducedMotion": true }  
}
```

IDE 골격

어두운 테마, 보라색

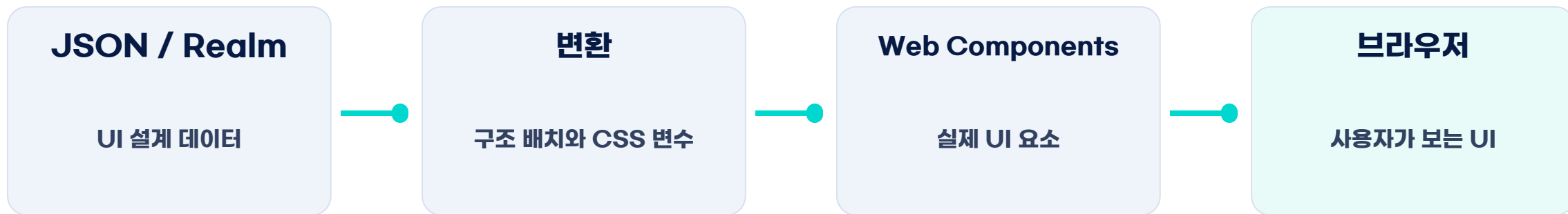
오른쪽 패널 숨김

움직임 줄이기



# Materialization이 선언을 웹 UI로 바꿉니다

Conjure-UI도 앞서 본 웹 실행 기반 위에서 작동합니다



**선택은 데이터로, 표현은 준비된 구성요소로**



# Conjure-UI를 웹 서비스에 적용했습니다

## NAIS ADME/Tox 서비스 적용 사례

같은 서비스에서  
디자인을 바꿉니다

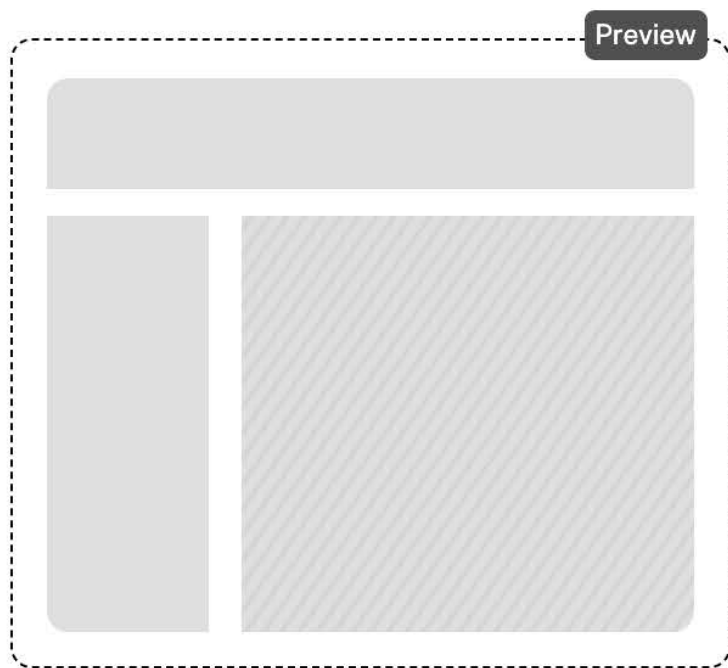
메뉴 배치와 색상을  
준비된 설정으로 조정합니다

The screenshot displays the NAIS ADME/Tox web service interface. On the left is a sidebar with a search bar and a menu containing 'Home', '자원 현황', 'AI 서비스' (with a sub-item 'ADME/Tox 예측'), 'AI 에이전트', and '관리'. The main area is titled 'ADME/TOX 예측' and includes a breadcrumb 'NAIS / AI 서비스 / ADME/Tox 예측'. It features a '분자 편집기·뷰어' (Molecule Editor/Viewer) section with tabs for '2D Molecule Editor' and '3D Viewer'. A toolbar with various editing tools is positioned above the central canvas, which displays a chemical structure of caffeine. To the right of the canvas is a vertical element palette listing atoms: H, C, N, O, S, P, F, Cl, Br, I, PT, and ET.

# 메뉴 높이를 설정으로 바꿉니다

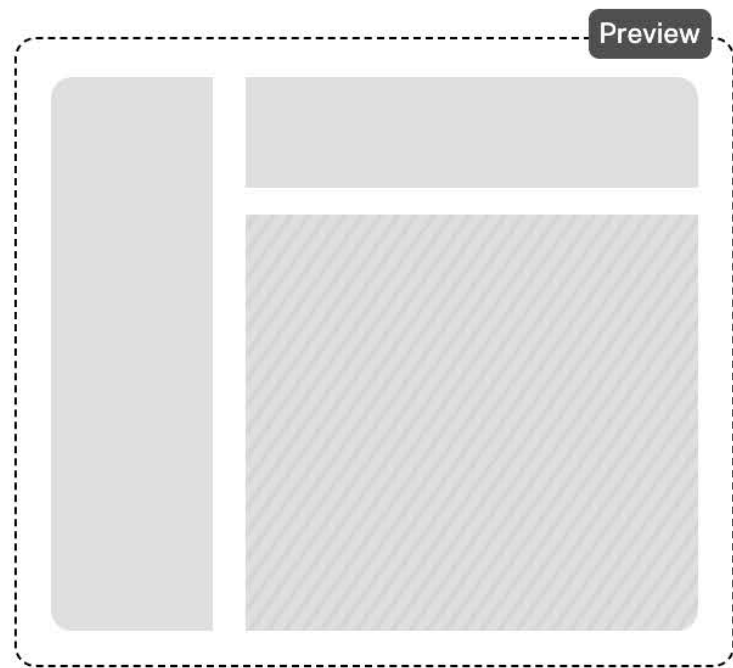
‘메뉴 전체 높이’ 선택에 따라 머리말과 메뉴의 배치가 달라집니다

## 기본 배치



☐ 메뉴 전체 높이

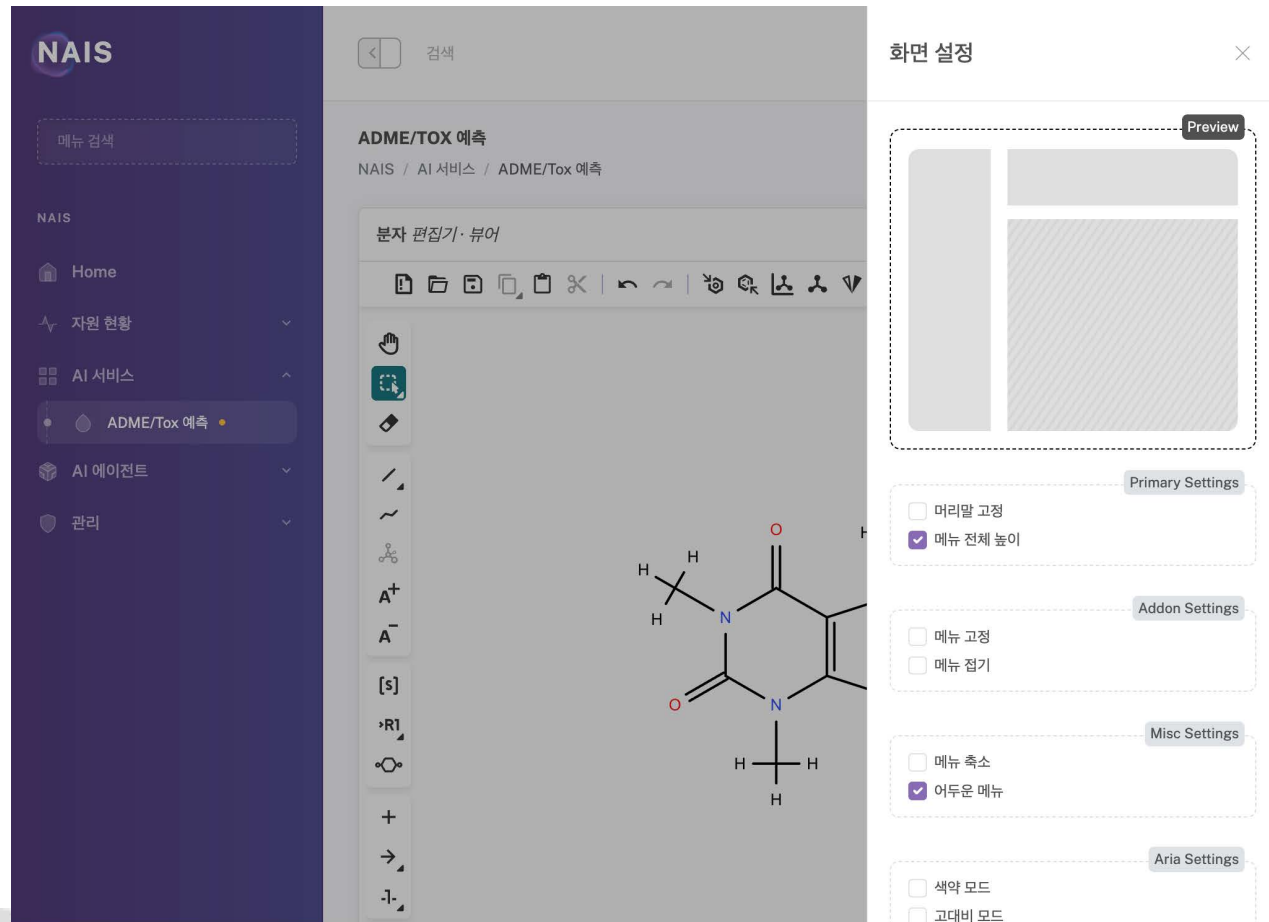
## 메뉴 전체 높이



☒ 메뉴 전체 높이

# 메뉴 색상도 설정으로 바꿉니다

‘어두운 메뉴’를 선택한 적용 예시



## 어두운 메뉴

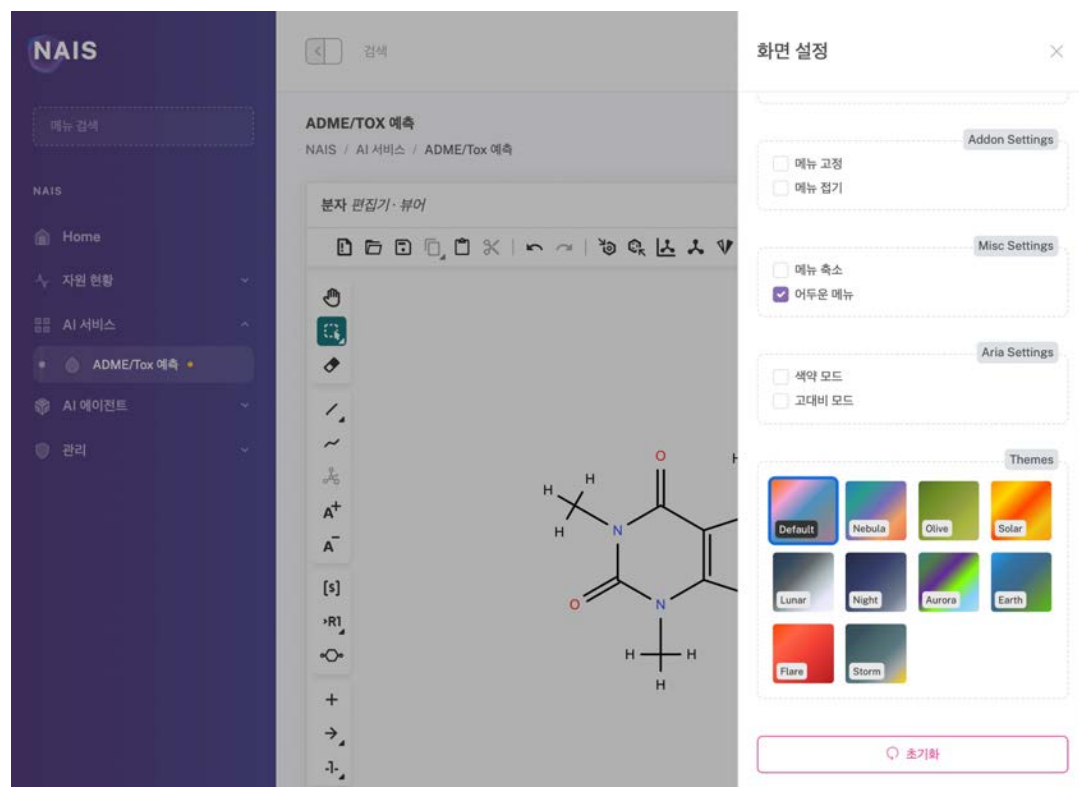
☒ 어두운 메뉴

메뉴 영역에  
어두운 색상을 적용

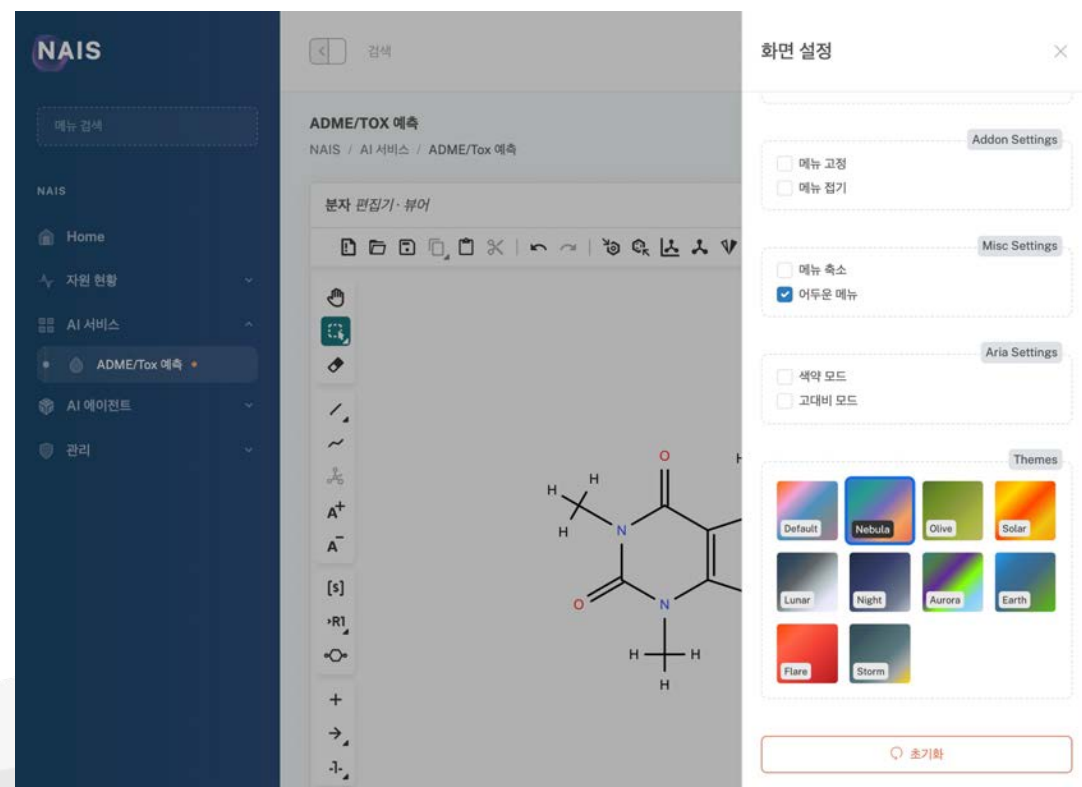
# 테마를 선택하면 색상 조합이 달라집니다

같은 '어두운 메뉴' 설정에서 Default와 Nebula를 비교합니다

## Default



## Nebula



# 핵심 모델과 프리셋을 구현하며 발전 중입니다

2026.06.26 발표자료에 제시한 구현 범위

**모델**



**Realm과 구조 교환 API**

**디자인**

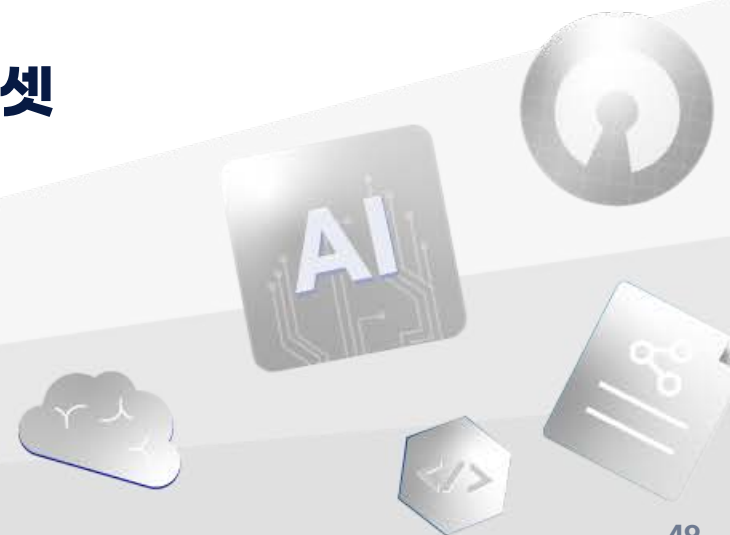


**Rune / Glamour와 CSS 변수 변환**

**UI 요소**



**Web Components와 8개 프리셋**



# 확장할 기능과 검증할 효과가 남아 있습니다

현재 구현과 다음 개발 목표를 구분합니다

## 개발 방향

제약 검사와 구조 확장  
재사용 자산을 모을 Registry

## 검증 과제

수정 후에도 규칙이 유지되는가  
오류와 수정 비용이 줄어드는가



# 목표는 반복 선택을 줄여, 수정을 쉽게 하는 것

Conjure-UI에서 시도하고 있는 방향

## 미리 정할 것

반복되는 UI 골격과  
디자인의 기본값

## 개발에 남길 것

앱의 목적과 기능,  
필요한 예외

이 질문은 Rails의 CoC와도 이어집니다



확인된 사실

# Rails는 반복 결정을 기본값으로 옮겼습니다

Convention over Configuration

Book Rails convention books

필요하면 **override**할 수 있지만, 대부분은 다시 정하지 않습니다



# 매번 결정하지 않아도 되는 자유

---

반복되는 선택을 기본값에 맡길수록 중요한 선택에 집중할 수 있습니다.



## 같은 질문, 서로 다른 메커니즘

이 결정을 매번 다시 내려야 하는가?

### Rails

암묵적 관례

반복 설정 감소

### AGENTS.md

작업 지침

반복 설명 감소

### DESIGN.md

지속적 디자인 설정

반복 디자인 결정 감소

### Conjure-UI

제한된 선언 언어

반복 UI 결정 감소



# 디자인 시스템은 CoC와 같지 않습니다

같은 문제의식을 다른 방식으로 해결합니다

## Rails CoC

Convention  
설정을 줄이는 기본값

## Design System

Persistent Configuration  
생성 결정을 줄이는 지속 설정

공통점은 좋은 결정을 다시 사용하는 것



# 관례, 검사, 권한은 서로 다른 장치입니다

설명했다고 강제된 것은 아닙니다

**관례**

어떻게 하는 것이 기본인가

경로를 제시

**검사**

결과가 규칙을 지켰는가

어긋남을 발견

**권한**

무엇을 실행할 수 있는가

행동을 제한



# 다시 부산행, 이번에는 실행 경계입니다

CoC의 기본값과 실행을 막는 안전장치는 구분해야 합니다

## 자동차

경로 선택이 넓다  
예외 대응이 쉽다  
잘못된 진입도 가능하다

## 기차

선로가 선택을 줄인다  
반복 운행이 예측 가능하다  
잘못 깔린 선로는 반복 실패한다



# 실행 경계는 네 겹으로 설계합니다

여러 장치를 함께 두어 한 번의 실수가 번질 가능성을 낮춥니다

**차단**

금지된 실행을 막게

**감지**

어긋나면 멈추게

**격리**

영향 범위를 가두게

**복구**

되돌릴 수 있게



# 좋은 제약은 선택과 책임을 분명히 합니다

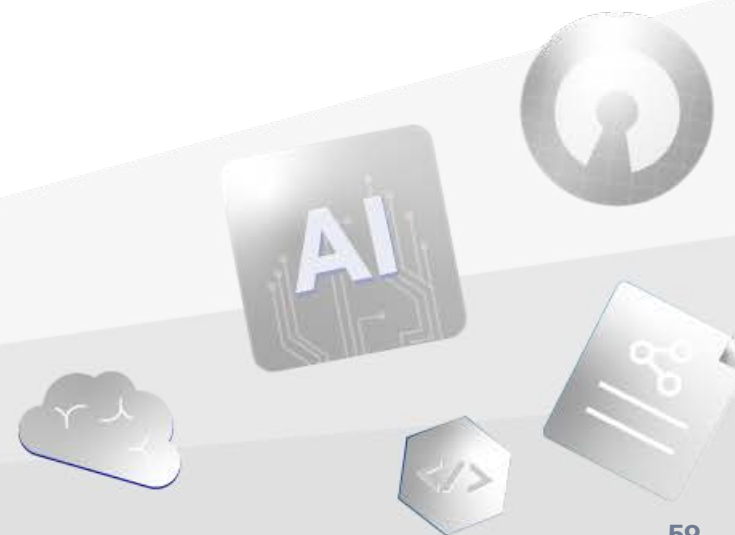
반복 선택은 줄이고, 안전 경계는 유지합니다

반복 결정 ————— 기본값으로 이동

핵심 판단 ————— 사람에게 남김

예외 ————— 승인하고 기록

실패 ————— 검사하고 되돌림



# 과학에서도 설명과 실행을 구분합니다

설명과 계획을 실제 실행 결과와 구분합니다

## 언어모델

---

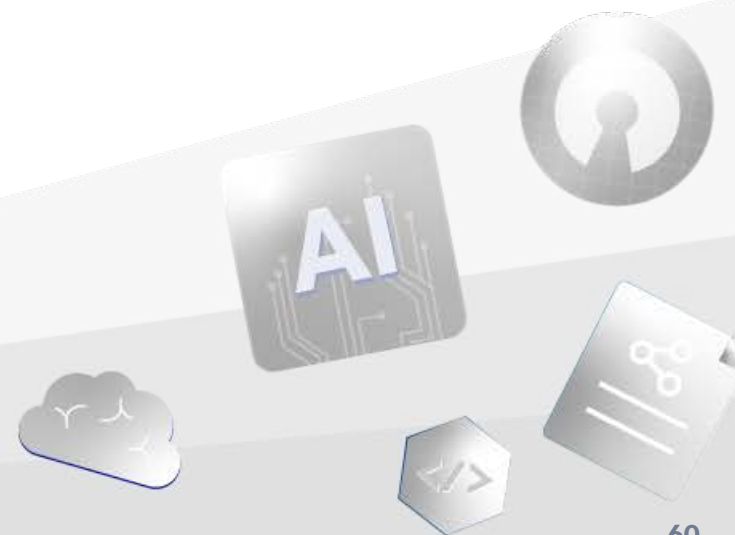
의도 해석  
작업 계획과 도구 호출

## 실행·검증 시스템

---

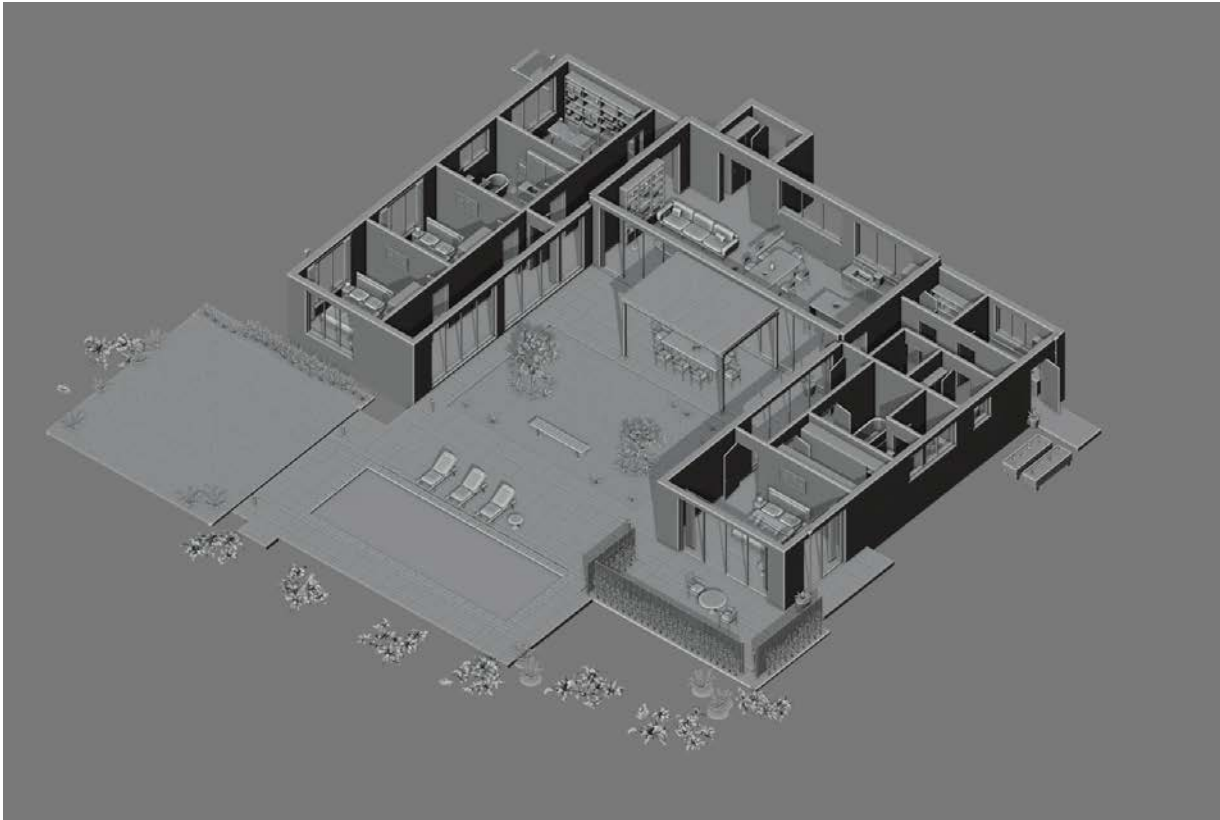
정해진 절차로 실행  
실행 기록과 결과 확인

설득력 있는 설명만으로 결과를 확정하지 않습니다



# 다시 보는 Astra와 Blender

언어모델의 계획을 프로그램의 실행과 검사로 연결한 공개 사례



3D 시각화 사례이며, 시공 적합성을 검증한 사례는 아닙니다

## Astra

작업 계획과 Python 코드

## Blender

장면 편집과 렌더 실행

## 확인

장면 구조와 렌더 점검



# RDKit은 정해진 화학 규칙으로 입력을 검사합니다

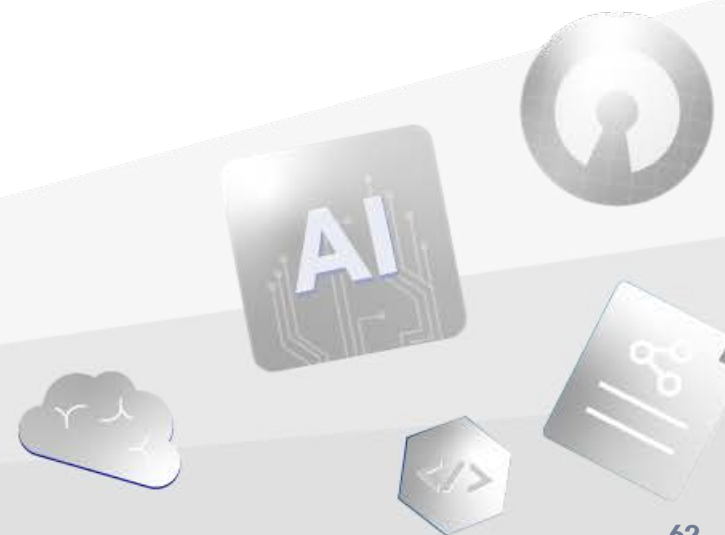
분자 구조를 읽을 때 수행하는 기본 원자가 검사



**규칙을 코드로 만들면 검사를 반복할 수 있습니다**

검사 통과가 합성 가능성이나 약효를 보장하지는 않습니다

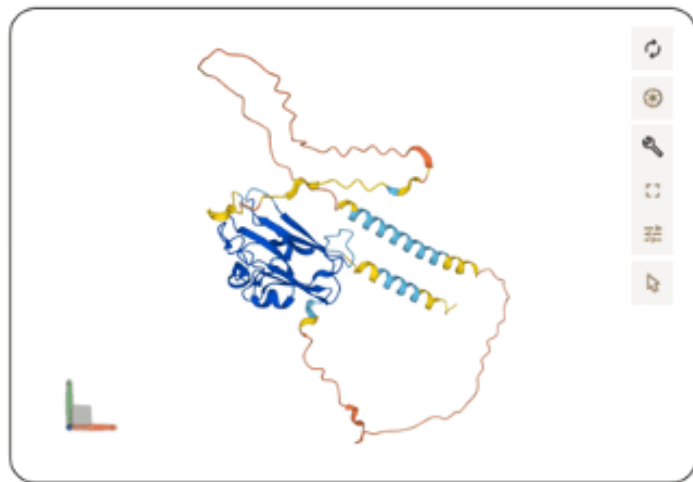
공식 문서에 근거한 동작 설명



# AlphaFold는 구조와 신뢰도를 함께 예측합니다

단백질 구조 예측 도구, AlphaFold 2의 pLDDT 예시

Ephrin-B2



Model Confidence ⓘ

■ Very high (pLDDT > 90)

■ Low (70 > pLDDT > 50)

■ High (90 > pLDDT > 70)

■ Very low (pLDDT < 50)

전용 예측 모델

3D 구조  
+ 신뢰도 정보

실험 검증 필요

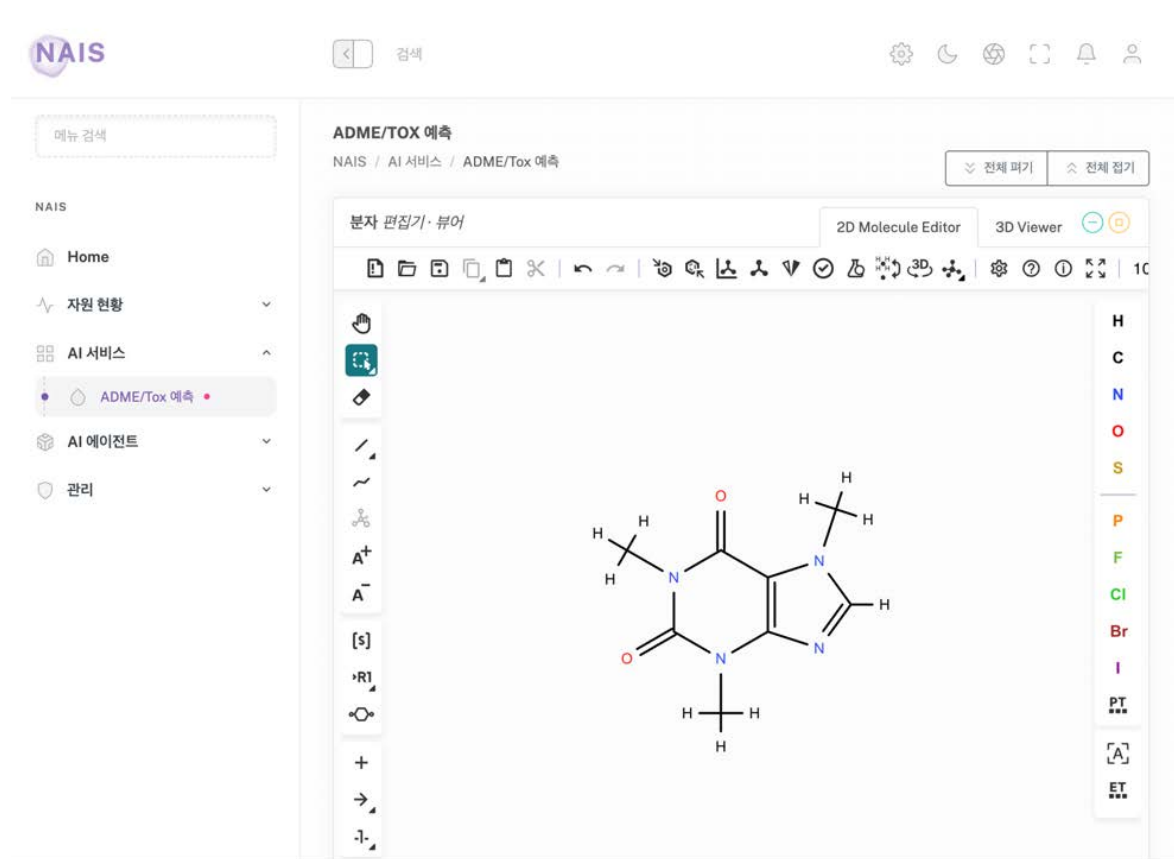
신뢰도가 낮은 구간은 해석에 주의합니다

EMBL-EBI Training · Figure 14 · CC BY 4.0



# NAIS의 ADME/Tox 예측 모델

Conjure-UI로 구성한 웹 서비스의 과학 도구 사례



NAIS ADME/Tox 서비스의 분자 입력 화면

전용 예측 모델

분자 구조를 입력받아  
ADME/Tox 특성 예측

언어모델 연계 제안

질문과 호출을 정리하고  
예측 결과 설명을 보조

예측값은 실험으로 확인한  
사실과 구분합니다

# 실행의 정확성과 과학적 타당성은 다릅니다

규칙 기반 도구와 학습 기반 예측 모델 모두 확인할 범위가 있습니다

## 실행의 정확성

---

**정한 알고리즘을  
의도대로 수행했는가?**

입력 조건과 구현 테스트

**도구의 실행 성공만으로 정답을 확정하지 않습니다**

## 과학적 타당성

---

**그 계산과 예측이  
현실을 설명하는가?**

모델의 가정과 실험 검증



# 언어모델의 도구 호출도 규칙으로 제한합니다

안전한 과학 프로그램을 위한 실행 흐름 제안



**검사에 실패하면 중단하고 기록합니다**

**실행 경계와 검증 기준은 프로그램이 관리합니다**



# 오픈소스로 공유할 실행·검증 묶음

오픈소스로 함께 다듬을 수 있는 다음 단위

## 규칙

schema · convention  
s

## 검사

lint · tests

## 권한

safe defaults

## 복구

snapshot · rollback

실행 방법과 실패 처리까지 재사용 가능한 형태로 공유합니다



# 과학 도구와 라이브러리 개발도 안전한 AI 활용의 기반입니다

---

분야 지식을 실행 가능한 규칙과 검사로 공유합니다

검증용 데이터와 실패 처리까지  
오픈소스로 함께 발전시킵니다



# 더 많이 말기려면

## 매번 판단하지 않아도 될 일과, 틀려도 지켜질 경계를 먼저 설계합니다

---

감사합니다

